

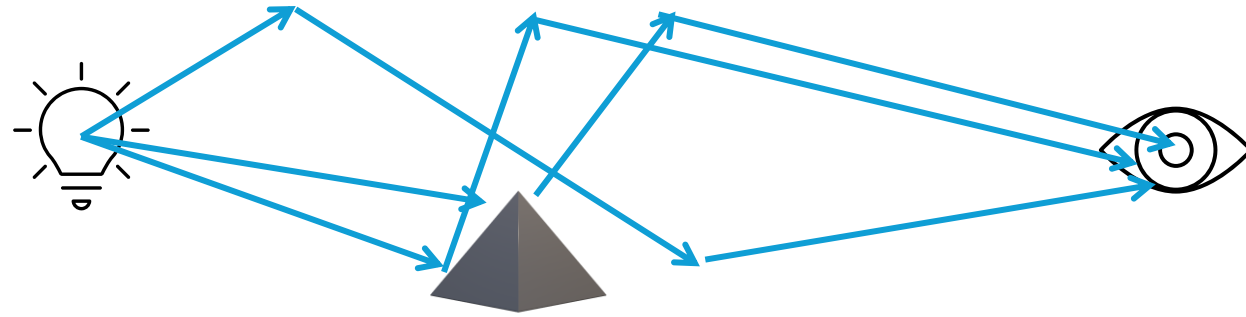
The background of the slide is a photograph of a historic building, likely a university building, with a prominent clock tower. The building is made of light-colored stone and has many windows. There are large trees in the foreground, some with green leaves and some with yellowing leaves, suggesting autumn. A semi-transparent white box is overlaid on the left side of the image, containing the title and presenter information.

3D Render - Basics

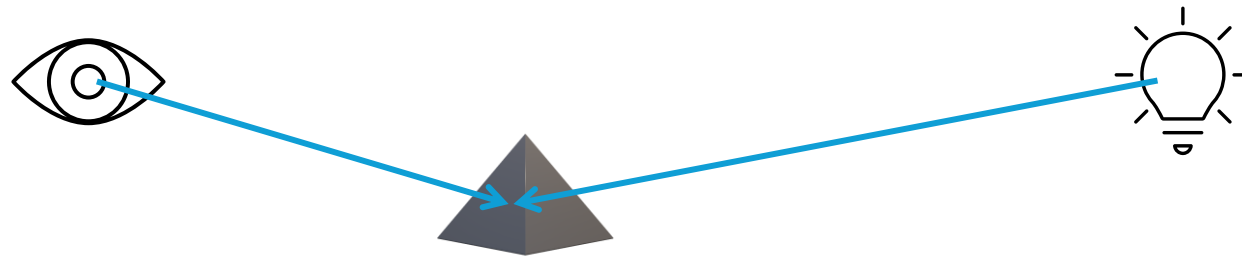
Dr. Torsten Bosse
Lehrstuhl für Advanced Computing
FSU Jena

The Core Concept: Ray Tracing

- **"Forward"**: Light bounces off objects into our eyes (too many rays to track!).



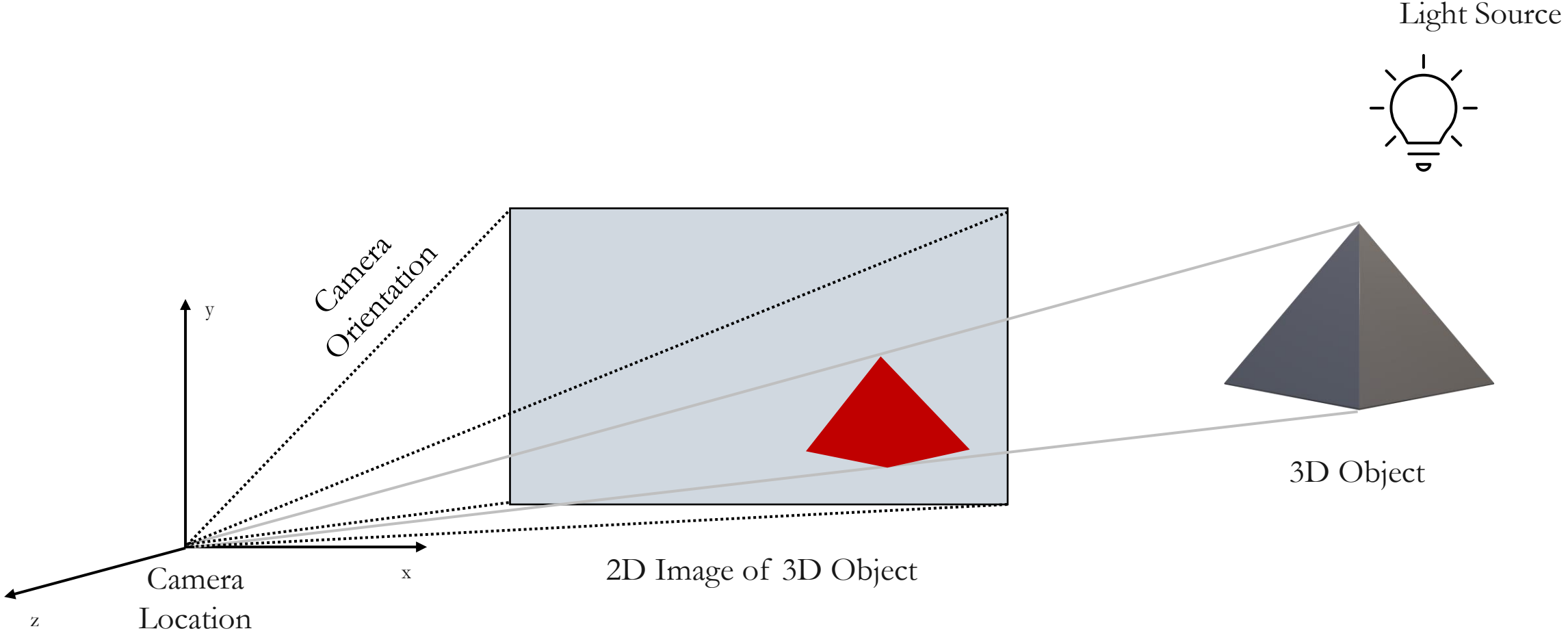
- **"Inverse"**: Shoot rays *from* eye, through a pixel, into the scene.



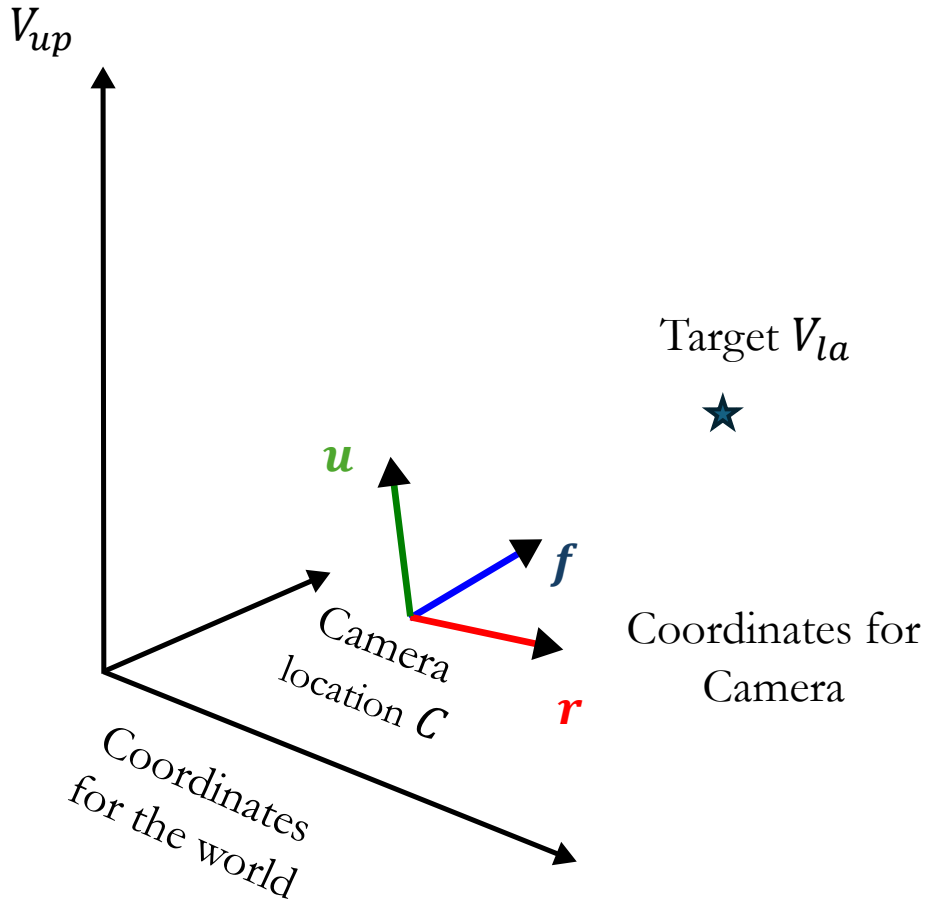
Idea: Find first object the ray hits and calculate its colour based on a light source.

→ Setup? Ray? Object? Hits? First Hit? Shading/Lighting? ...

Camera Scene



Creating the Camera Basis



To define the camera's orientation in $\mathbb{R}^3$, derive a local coordinate system based on the camera location C , a target point V_{la} (look at), and a vector V_{up} pointing “upwards”.

Forward Vector (f): Defines the primary optical axis.

$$f = \frac{V_{la} - C}{\|V_{la} - C\|}.$$

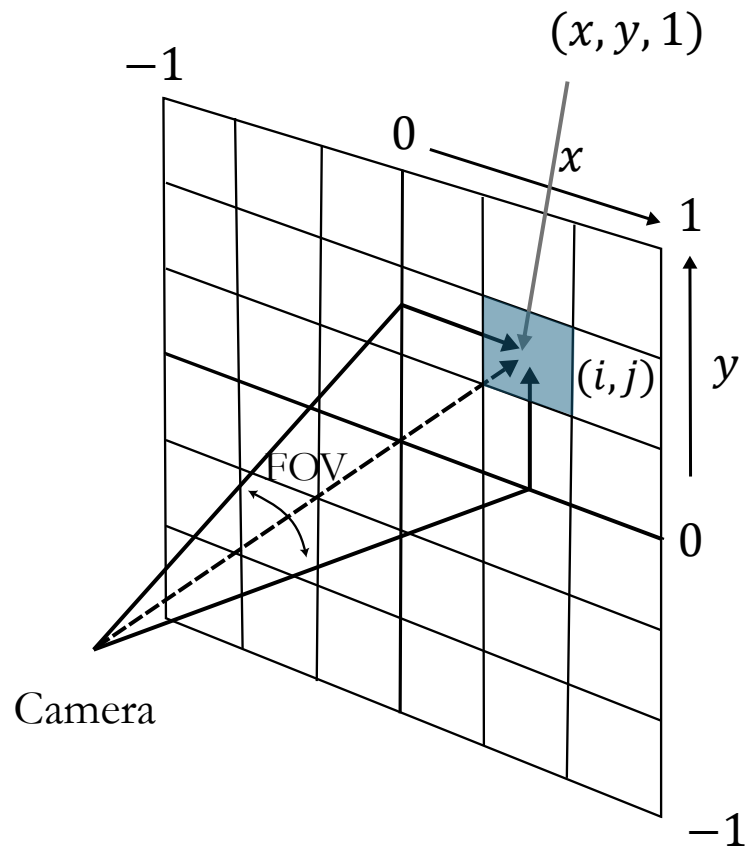
Right Vector (r): Determines the horizontal axis of the image plane, orthogonal to the optical axis and the world-up direction.

$$r = \frac{f \times V_{up}}{\|f \times V_{up}\|}.$$

Camera-Up Vector (u): Ensures a perfect right-handed coordinate system by calculating the vertical axis orthogonal to both f and r .

$$u = r \times f.$$

Mapping Pixels to the Film Plane



Map discrete pixel coordinates (i, j) from the image sensor of size $(W \times H)$ to continuous coordinates (x, y) coordinates ranging from -1 to 1 on a virtual image plane situated at distance $d = 1$ from the focal point. This mapping accounts for the **Field of View** θ and **Aspect Ratio** a .

The local coordinates x and y are calculated as:

$$x = \left(\frac{2(i+0.5)}{W} - 1 \right) \cdot a \cdot \tan\left(\frac{\theta}{2}\right)$$

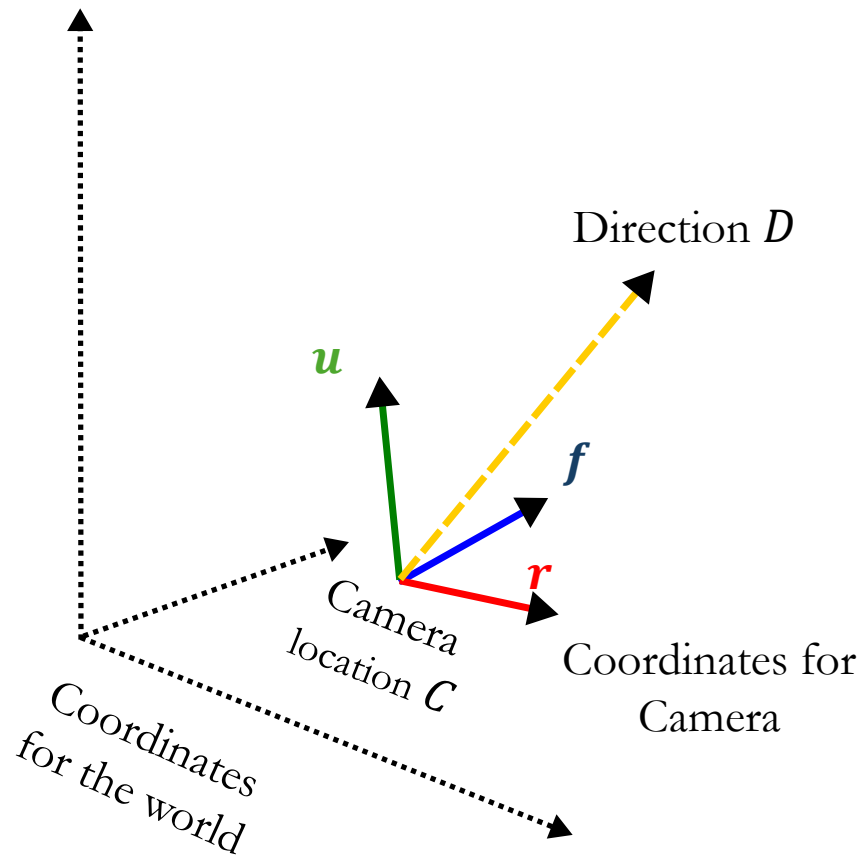
$$y = \left(\frac{2(j+0.5)}{H} - 1 \right) \cdot \tan\left(\frac{\theta}{2}\right)$$

Combining into World Direction

The ray direction D is computed as a linear combination of the orthonormal basis vectors. This effectively rotates and aligns the 2D image plane coordinates into the 3D world environment:

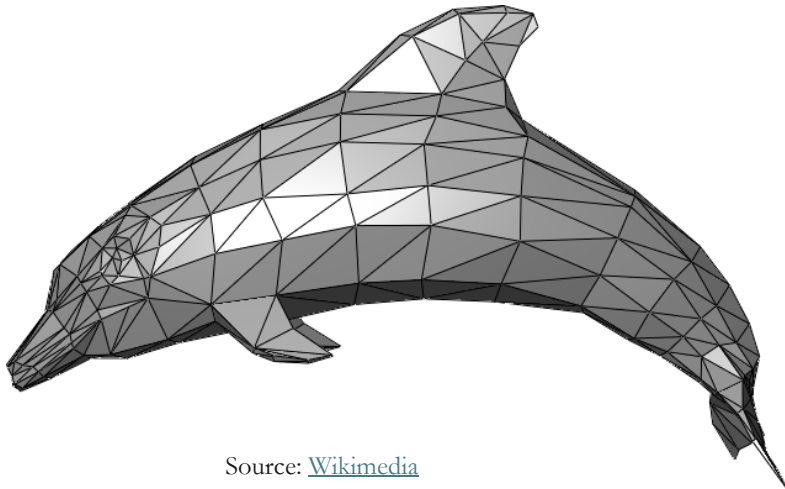
$$D = \text{normalize}(xr + yu + f)$$

By utilizing this vector-based approach, the ray-tracer becomes invariant to the global coordinate system's orientation provided the basis is correctly initialized.

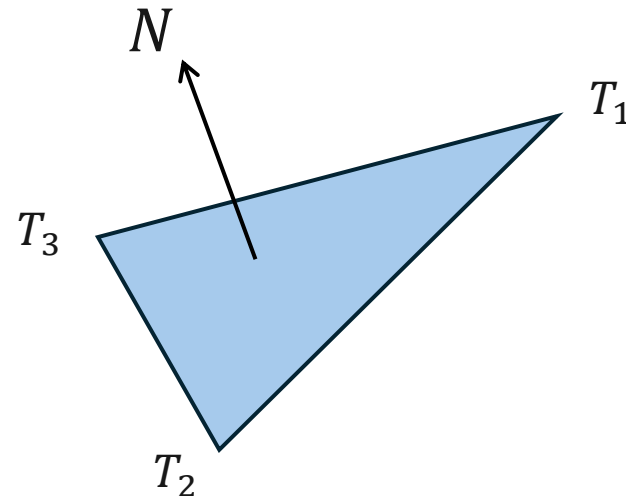


Representation of 3D Objects

- **3D Objects:** Typically described by a mesh of triangles $T \subseteq \mathbb{R}^3$.
- **Triangle:** Fundamental building block described by its corners $T_1, T_2, T_3 \in \mathbb{R}^3$.
- **Normal:** Provides orientation of triangle and is given by $N = (T_2 - T_1) \times (T_3 - T_1)$.



Source: [Wikimedia](#)

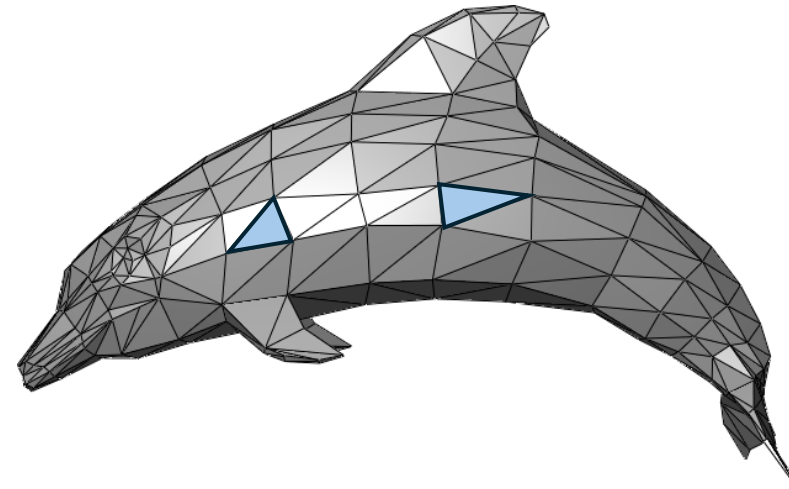


STL-Files for 3D Objects

Standard file format to store 3D meshes as a list of triangles (with normal directions).

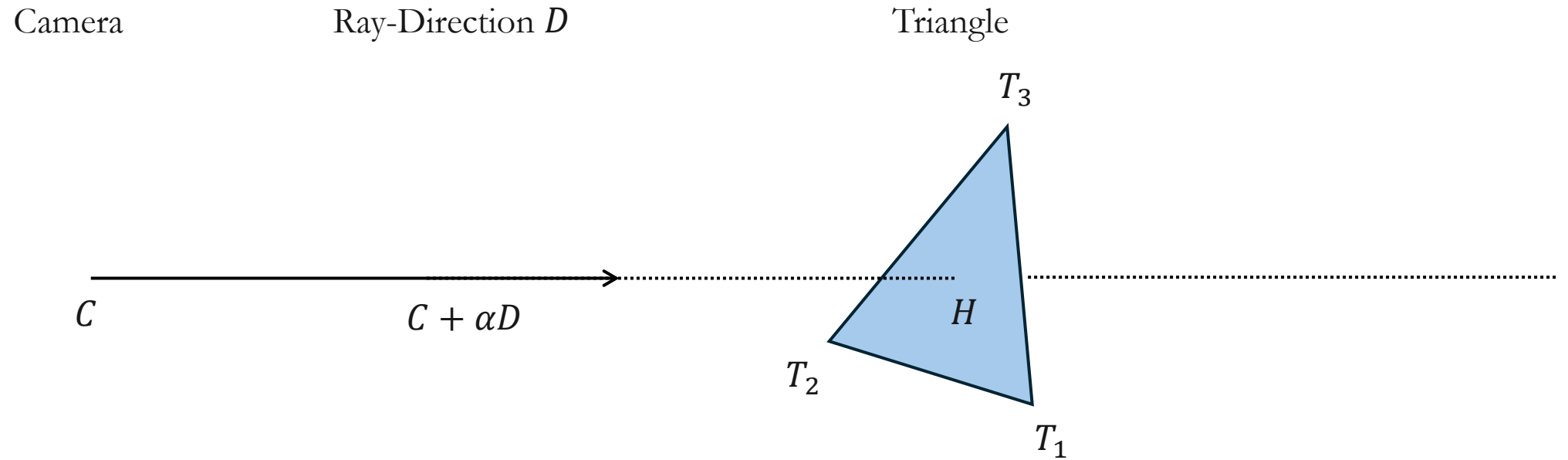
ASCII -Example.

```
solid ASCII_STL_of_a_dolphin
facet normal 0.70675 -0.70746 0
  outer loop
    vertex 1000 0 0
    vertex 0 -1000 0
    vertex 0 -999 -52
  endloop
endfacet
facet normal 0.05229 -0.73152 0.67981
  outer loop
    vertex 1000 0 0
    vertex 0 -999 -52
    vertex 0 -995 -105
  endloop
endfacet
...
```



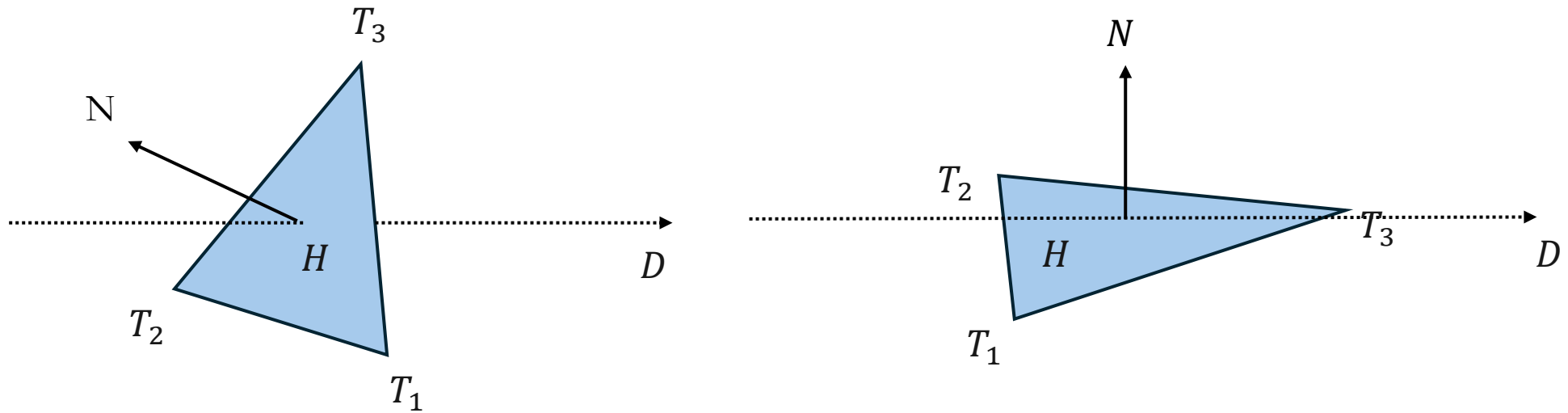
Source: [Wikimedia](#)

Triangle



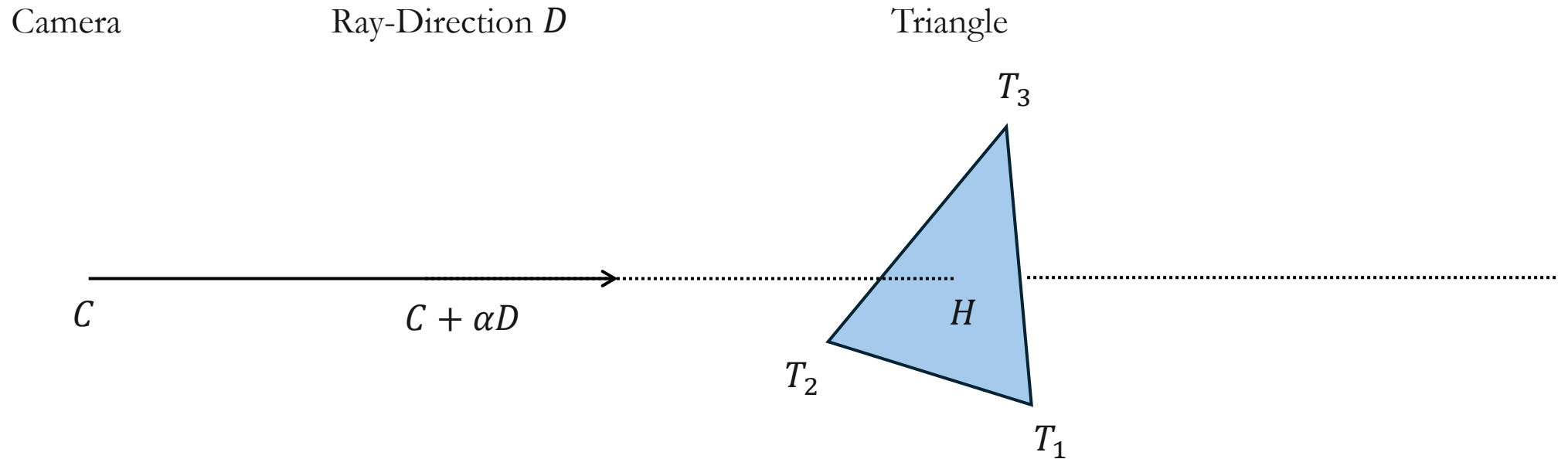
Given a ray with origin $C \in \mathbb{R}^3$ and direction $D \in \mathbb{R}^3$, determine the intersection point $H \in \mathbb{R}^3$ of the ray with the triangle spanned by T_1 , T_2 , and T_3 , if existing.

Triangle Orientation



Intersection H is unique if triangle is not parallel to ray, namely, ray direction D is not orthogonal to the triangle's normal $N = (T_3 - T_1) \times (T_2 - T_1)$, i.e., $\beta = \langle N, D \rangle \neq 0$.

Triangle Intersection



Intersection point $H \in \mathbb{R}^3$ on the triangle is given by convex combination of T_1 , T_2 , and T_3 , namely, $H = \lambda_1 T_1 + \lambda_2 T_2 + \lambda_3 T_3$ with $0 \leq \lambda_1, \lambda_2, \lambda_3$ and $\lambda_1 + \lambda_2 + \lambda_3 = 1$ or, equivalently,

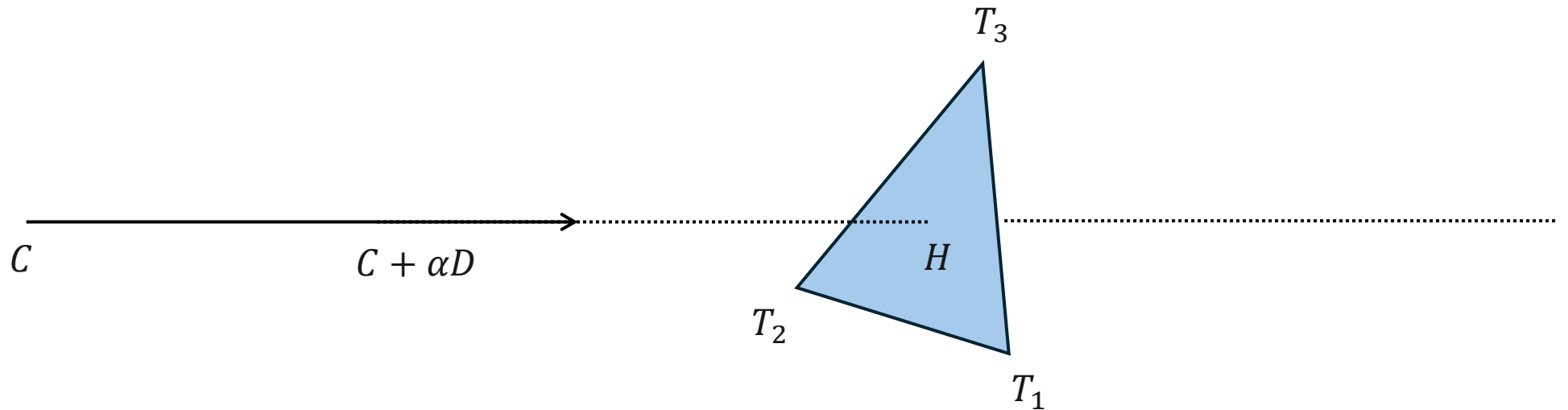
$$H = (1 - \lambda_2 - \lambda_3)T_1 + \lambda_2 T_2 + \lambda_3 T_3 \text{ with } 0 \leq \lambda_2, \lambda_3$$

Triangle Intersection

Camera

Ray-Direction D

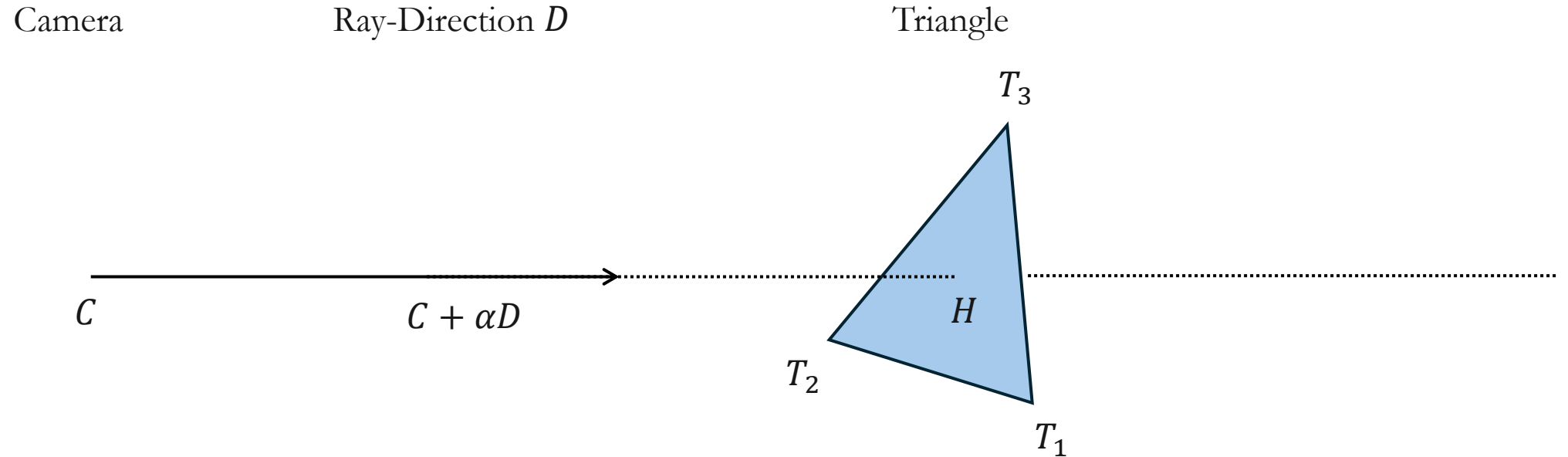
Triangle



$$C + \alpha D = (1 - \lambda_2 - \lambda_3)T_1 + \lambda_2 T_2 + \lambda_3 T_3$$
$$\Leftrightarrow C - T_1 = \lambda_2(T_2 - T_1) + \lambda_3(T_3 - T_1) - \alpha D$$

$$\Leftrightarrow \begin{bmatrix} \vdots & \vdots & \vdots \\ T_2 - T_1 & T_3 - T_1 & -D \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \lambda_2 \\ \lambda_3 \\ \alpha \end{bmatrix} = \begin{bmatrix} \vdots \\ C - T_1 \\ \vdots \end{bmatrix}$$

Triangle Intersection



$$\begin{bmatrix} \vdots \\ T_2 - T_1 \\ \vdots \end{bmatrix} \begin{bmatrix} \vdots \\ T_3 - T_1 \\ \vdots \end{bmatrix} \begin{bmatrix} \vdots \\ -D \\ \vdots \end{bmatrix} \begin{bmatrix} \lambda_2 \\ \lambda_3 \\ \alpha \end{bmatrix} = \begin{bmatrix} \vdots \\ C - T_1 \\ \vdots \end{bmatrix}$$

H is **on** the triangle if and only if the solution satisfies $0 \leq \lambda_1, \lambda_2$ and $\lambda_1 + \lambda_2 \leq 1$.

Cramer's Rule for Solving the 3x3 System

Finding the values λ_2 , λ_2 , and α that satisfy the equation

$$\begin{bmatrix} \vdots & \vdots & \vdots \\ T_2 - T_1 & T_3 - T_1 & -D \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \lambda_2 \\ \lambda_3 \\ \alpha \end{bmatrix} = \begin{bmatrix} \vdots \\ C - T_1 \\ \vdots \end{bmatrix}$$

corresponds to solving the standard linear 3x3 system

$$\begin{bmatrix} \vdots & \vdots & \vdots \\ A_1 & A_2 & A_3 \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \vdots \\ b \\ \vdots \end{bmatrix}.$$

The solution of this system (if existing and unique) is given by Cramer's rule using the following determinant equivalence

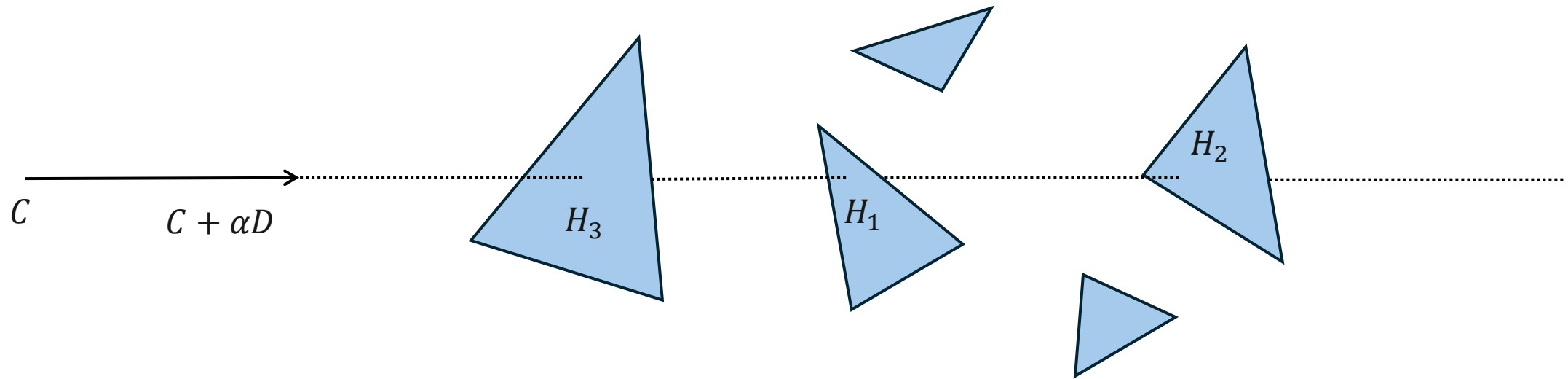
$$\det[M] = \det \begin{bmatrix} \vdots & \vdots & \vdots \\ M_1 & M_2 & M_3 \\ \vdots & \vdots & \vdots \end{bmatrix} = \langle M_1, M_2 \times M_3 \rangle.$$

$$x_1 = \frac{1}{\det[A]} \det \begin{bmatrix} \vdots & \vdots & \vdots \\ b & A_2 & A_3 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

$$x_2 = \frac{1}{\det[A]} \det \begin{bmatrix} \vdots & \vdots & \vdots \\ A_1 & b & A_3 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

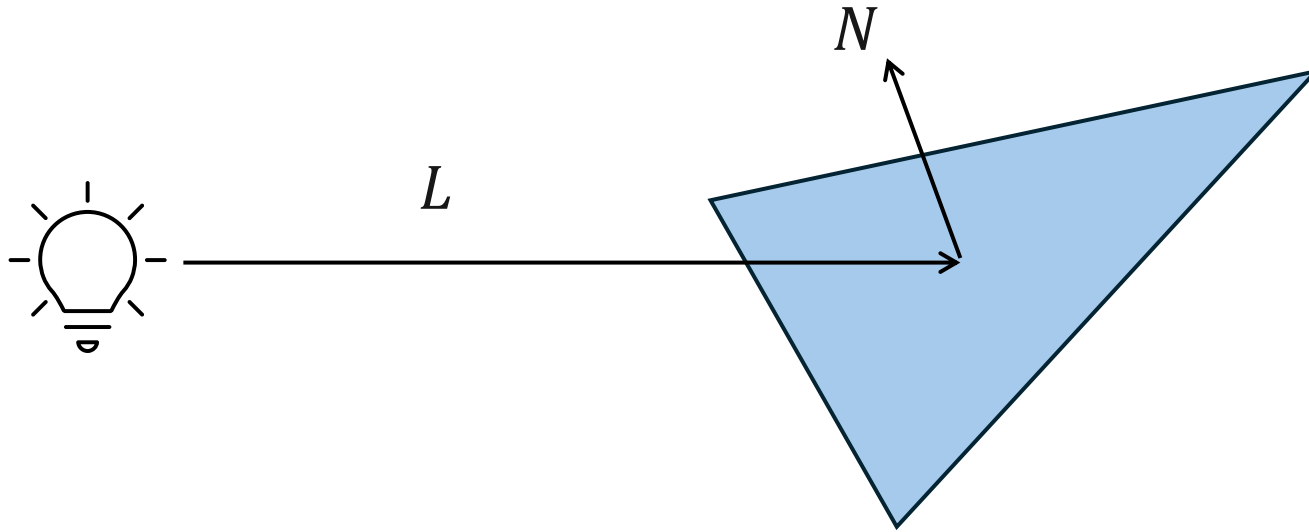
$$x_3 = \frac{1}{\det[A]} \det \begin{bmatrix} \vdots & \vdots & \vdots \\ A_1 & A_2 & b \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Möller–Trumbore Intersection Algorithm



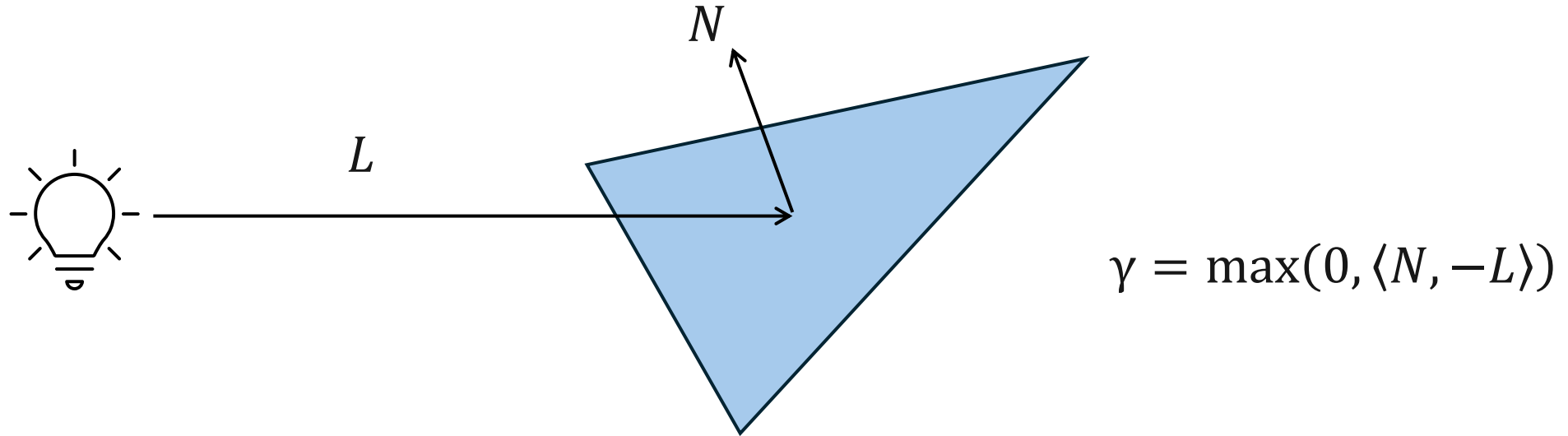
IDEA: For given ray $C + \alpha D$ determine nearest triangle, i.e., compute for all triangles the intersection point (if existing and unique) and chose the one with smallest corresponding (non-negative) scaling value α .

Lambertian (diffuse) lighting calculation



IDEA: Determine for (nearest) triangle with normal N its illumination based on the direction for a light coming from a source, i.e., determine how aligned the surface normal and light direction L are via inner product.

Lambertian (diffuse) lighting calculation



Measure how aligned the surface normal and light direction are:

- $\langle N, -L \rangle = 1 \rightarrow$ fully facing the light
- $\langle N, -L \rangle = 0 \rightarrow$ perpendicular (grazing angle)
- $\langle N, -L \rangle < 0 \rightarrow$ facing away from the light

Rendering Pipeline

Define Scene

Read STL File

For every pixel $P_{i,j}$ in image:

- 1.Direction:** Calculate the ray from the camera through the pixel.
- 2.Trace loop:** For all triangles, determine the one with closest ray intersection using MTI algorithm and Cramer's Rule.
- 3.Shade:** For closest triangle calculate the color based normal and light direction.
- 4.Store:** Convert and store the float color values as integer values (0-255).

```
def render(...):
    forward = (cam_lookat - C).normalize()
    ...
    scene = Scene()

    tris = load_stl(stl_file, Vec3(0.8, 0.8, 0.8))
    scene.add_triangles(tris)

    pixels: List[Tuple[int, int, int]] = []
    for j in range(height):
        for i in range(width):
            x = (2 * (i + 0.5) / width - 1) * math.tan(fov / 2) * aspect
            y = -(2 * (j + 0.5) / height - 1) * math.tan(fov / 2)
            D = (right * x + actual_up * y + forward).normalize()
            color = scene.trace(C, D, light)
            r = max(0, min(255, int(255 * color.x)))
            g = max(0, min(255, int(255 * color.y)))
            b = max(0, min(255, int(255 * color.z)))
            pixels.append((r, g, b))

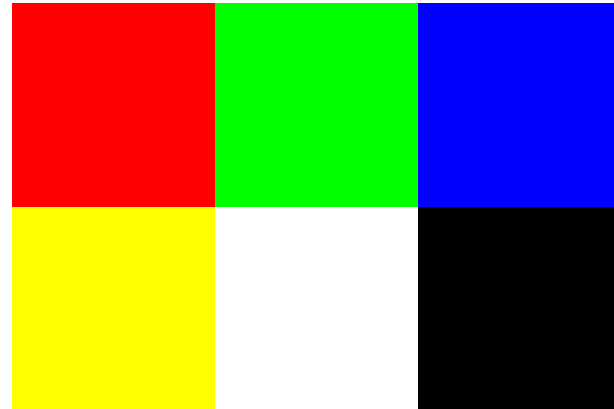
    write_ppm("output.ppm", width, height, pixels)
    show_image(width, height, pixels)
```


The Final Output: PPM Format

PPM-files are a simple format to store images as pixel graphics with RGB colors.

PPM -Example

```
P3
# "P3" is a RGB color image in ASCII
# The width and height is set by "3 2"
# "255" is the max. value for each color
# This, up through the "255" line below
# is the header and everything after that
# the image data stored as RGB triplets.
3 2
255
255 0 0 0 255 0 0 0 255
255 255 0 255 255 255 0 0 0
```



Source: [Wikimedia](#)

Conclusion



- **Key Takeaway:** You don't need a heavy engine to understand the fundamentals.
- **Algorithm:** 3D rendering is just geometry and linear algebra applied at scale.
- **Complexity:** $O(\text{Pixels} * \text{Triangles})$.
- **Bottleneck:** Checking every triangle for every pixel is slow for high-poly models.
- **Multithreading:** Trace different pixels/rays on different CPU cores.
- **Further ideas:** Only check triangles near the ray (see Spatial Partitioning/BVH).

Questions?